

Dodávka on-premises produktu

Model divergence stavu a mechanismy jeho zúžení

Jiří Slachta

Pracovní verze (draft)

draft-b5073c5 — commit b5073c5 — 2026-10-02

Abstrakt

Text popisuje dodávku software provozovaného v prostředí zákazníka jako problém řízení divergence: rozdílu mezi skutečným stavem instance a představou dodavatele o tomto stavu. Ukazuje, že náklad této formy dodávky neroste s rychlostí vývoje, ale s počtem živých kombinací, které musí dodavatel obsloužit, a že většina běžně doporučovaných opatření (pomalá kadence, minimalizace počtu vydání) tento počet ve skutečnosti zvyšuje. Jádrem je model cestové závislosti stavu databáze a z něj odvozená sada invariantů a mechanismů.

Obsah

1	Rámec	1
2	Model	1
2.1	Dva stavy	1
2.2	Verze schématu není stav systému	2
2.3	Konvergentní a divergentní transformace	2
2.4	Podmínky bezpečnosti divergentní transformace	2
2.5	Zdroje nedeterminismu, které projdou code review	3
2.6	Dvě cesty k témuž schématu	3
2.7	Latence detekce jako hlavní nákladový faktor	4
2.8	Co z modelu plyne pro měřitelnost	4
3	Invarianty	4
4	Mechanismy	5
5	Rozhodnutí, která zůstávají úsudkem	5
6	Antipatterny	5
	Zdroje	6

1 Rámec

[Stub. Obsah: on-prem jako provozní režim, nikoli distribuční model. Rozlišení mezi tím, co skutečně zaniklo (ruční instalace bez pipeline), a tím, co přetrvává (regulace, OT, ekonomika

stabilní zátěže, inference nad citlivými daty). Nákladový driver: počet živých kombinací, nikoli rychlost vývoje. Vyvrácení intuice, že pomalá kadence zužuje pole verzí.]

2 Model

2.1 Dva stavy

Instance provozovaná u zákazníka má v každém okamžiku dva stavy, které je nutné držet odděleně.

Skutečný stav S je to, co je fyzicky v databázi a na disku: struktura schématu, obsah dat, přítomnost či nepřítomnost objektů, které tam kdo kdy vytvořil.

Evidovaný stav E je to, co si o instanci myslí dodavatel a co si o sobě myslí instalátor: číslo verze, obsah migračního ledgeru, záznamy v systémech dodavatele.

V ideálním případě platí $S \equiv E$. V praxi vzniká divergence $D = S \setminus E$ — množina skutečností, které v instanci platí, ale nikde nejsou evidované. Tato práce vychází z jediné teze:

**Selhání dodávky on-premises produktu nevzniká z chyb v kódu.
Vzniká tehdy, když se rozhoduje podle E , zatímco operace působí na S .**

Migrace napsaná proti evidovanému stavu spadne, protože je aplikována na stav skutečný. Rozhodnutí o vydání učiněné podle evidence o verzích v poli je chybné, protože pole vypadá jinak. Diagnóza vzdálené instance je chybná, protože se opírá o číslo verze, které nepopisuje to, co je uvnitř.

Všechny mechanismy popsané v sekci 4 mají jediný účel: zmenšit D , případně učinit ji pozorovatelnou dřív, než se o ni někdo opře.

2.2 Verze schématu není stav systému

Většina migračních nástrojů drží jediné číslo — kde jsem. Tím implicitně tvrdí, že stav je funkcí cíle:

$$S = f(v)$$

kde v je verze schématu. Toto tvrzení je nepravdivé. Stav je funkcí cesty:

$$S = f(S_0, \langle m_1, m_2, \dots, m_n \rangle)$$

kde S_0 je výchozí stav a $\langle m_i \rangle$ posloupnost skutečně aplikovaných transformací v pořadí, ve kterém proběhly.

Dvě instance na téže verzi tedy nemusí být v témže stavu. Instance, která prošla $v_3 \rightarrow v_4 \rightarrow \dots \rightarrow v_7$, a instance, která přišla jinou cestou nebo některé kroky přeskočila, mají shodné v a odlišné S .

Praktický důsledek: *prostor, který musí dodavatel testovat, není množina verzí, ale množina cest*. Ta roste podstatně rychleji a na rozdíl od množiny verzí není nikde evidovaná.

2.3 Konvergentní a divergentní transformace

Ne každá migrace přispívá k divergenci. Rozhodující je, zda je její výsledek závislý na vstupním stavu.

Konvergentní transformace má výsledek nezávislý na S i na cestě. Typicky čistě strukturální operace:

```
| ALTER TABLE WorkOrder ADD COLUMN ClosedAt datetime2 NULL;
```

Ať byla instance v jakémkoli stavu, po této operaci má sloupec. Cesty konvergují.

Divergentní transformace má výsledek, který je funkcí vstupního stavu:

```
| UPDATE WorkOrder
  SET ClosedAt = (SELECT MAX(ChangedAt) FROM WorkOrderLog l
                  WHERE l.WorkOrderId = WorkOrder.Id
                  AND l.NewState = 'Closed');
```

Výsledek závisí na tom, co bylo v `WorkOrderLog` v okamžiku běhu. Dvě instance se stejným schématem a odlišnou historií skončí s odlišnými daty. Cesty divergují.

Hranice tedy nevede mezi „malými“ a „velkými“ migracemi, ale mezi těmi, které se dotýkají pouze struktury, a těmi, které se dotýkají dat. Každá data-touching migrace je potenciální zdroj divergence a musí být posuzována jinou optikou.

2.4 Podmínky bezpečnosti divergentní transformace

Data-touching migraci lze učinit bezpečnou, pokud splňuje tři podmínky současně.

Determinismus

Nad týmž vstupem dává týž výstup. Zejména nezávisí na čase běhu, na lokálním nastavení instance ani na pořadí, ve kterém databáze vrátila řádky.

Idempotence

Opakované spuštění nezmění výsledek. Druhý běh se dotkne nula řádků.

Čistota vzhledem ke vstupnímu stavu

Výsledek je odvoditelný výhradně z dat, která jsou v okamžiku běhu v databázi. Nikoli z toho, jaká byla předchozí verze aplikace nebo který patch instance minula.

Pokud transformaci nelze takto napsat, není to migrace. Je to předpoklad, a předpoklad patří do preflightu jako ověřovaný invariant, nikoli do mutační fáze.

2.5 Zdroje nedeterminismu, které projdou code review

Nedeterminismus bývá v migracích neúmyslný a vizuálně nenápadný. Opakující se zdroje:

- **Aktuální čas.** `GETDATE()` v backfillu. Migrace spuštěná o rok později vyrobí jiná data než migrace spuštěná v den vydání.
- **Konfigurace instance.** Locale, časová zóna, collation, výchozí měna, výchozí jednotky. Tytéž řádky, jiný výsledek.
- **Objem dat v okamžiku běhu.** Zejména u transformací pracujících po dávkách, kde se počet dávek propisuje do výsledku.
- **Nedeterministické pořadí.** TOP nebo LIMIT bez úplného ORDER BY. Databáze nemá povinnost vrátit dvakrát totéž.
- **Volání aplikační logiky.** Migrace, která pro výpočet použije kód aplikace, je svázaná s verzí toho kódu. Ta se mění, migrace zůstává.

Společný jmenovatel: všechny tyto zdroje jsou neviditelné v okamžiku, kdy se migrace píše, protože autor ji spouští jednou, ve svém prostředí, ihned.

2.6 Dvě cesty k témuž schématu

Zvláštní případ cestové závislosti, který se projeví až po letech.

Nová instalace obvykle vzniká z baseline skriptu. Existující instance došla řetězem migrací. Obě jsou nominálně na téže verzi a obě jsou funkční. Liší se v detailech, které nikoho nezajímají, dokud na ně něco nesáhne:

- jména omezení — explicitní `PK_WorkOrder` proti autogenerovanému `PK__WorkOrder__3213E83F...`
- pořadí sloupců v tabulce
- existence indexů, které vznikly jako vedlejší efekt starší migrace a v baseline nejsou
- `IDENTITY` seedy a mezery v číselných řadách
- nullability a collation na úrovni sloupce, kde se chování `ALTER` liší od `CREATE`

Pak přijde migrace obsahující `DROP CONSTRAINT [FK_WorkOrder_Asset]` a selže u té poloviny pole, která má omezení pojmenované jinak.

Tento případ je uveden zvlášť, protože je na rozdíl od ostatních *levně a úplně automatizovatelně detekovatelný* — viz sekce 4.

2.7 Latence detekce jako hlavní nákladový faktor

Divergence se neprojeví v okamžiku vzniku. Projeví se při první operaci, která se o porušení předpokládá opřít. Mezi těmito dvěma okamžiky uplyne libovolně dlouhá doba.

To má tři důsledky, které určují ekonomiku celé dodávky:

1. **Náklad opravy roste s latencí.** Divergence odhalená před upgradem je konfigurační problém. Táž divergence odhalená během upgradu je havárie s odstávkou. Odhalená tři měsíce po upgradu je forenzní úloha nad poškozenými daty.
2. **Příčina se s latencí ztrácí.** Po dostatečné době není zjistitelné, která migrace stav porušila, protože chybí záznam o tom, co bylo předtím.
3. **Divergence se skládá.** Neopravená divergence se stává vstupním stavem pro další transformace, které s ní nepočítají.

Cílem tedy není nulová divergence — ta je nedosažitelná. Cílem je **krátká latence detekce**: aby se rozpor mezi *S* a *E* projevil v okamžiku, kdy je jeho oprava levná, a nikoli v okamžiku, kdy je drahá.

2.8 Co z modelu plyne pro měřitelnost

Aby byla latence krátká, musí být divergence pozorovatelná. Z předchozích odstavců plynou tři veličiny, které musí být zaznamenatelné a přenositelné:

Cesta, nikoli verze

Ledger aplikovaných transformací: co, kdy, jakou verzí nástroje, s jakým výsledkem, s jakým otiskem skriptu v okamžiku aplikace. Otisk je nutný, protože skript opravený po vydání dělí pole na instance před opravou a po ní.

Otisk dat, nikoli jen schématu

Kontrolní součty nad klíčovými tabulkami, distribuce hodnot výčtových sloupců, počty porušení jednotlivých invariantů, seznam objektů v databázi, které nepocházejí od dodavatele.

Výsledek nezávislého ověření

Tvrzení o platnosti invariantů získané dotazem nad daty, nikoli odvozené z evidence o tom, že migrace proběhla.

Poslední bod je podstatou celého modelu. Evidence o provedení operace není důkazem o jejím výsledku.

3 Invarianty

[Stub. Číslovaná, citovatelná sada. Každý invariant: jedna věta tvrzení, odstavec zdůvodnění odkazující do sekce 2, poznámka o tom, jak se ověřuje. Číslo je stabilní a nemění se při přeuspořádání textu.]

INV-01 Schéma se v rámci patch řady nemění.

[Zdůvodnění, výjimka pro aktivně destruktivní stav, způsob ověření.]

INV-02 Data-touching migrace je deterministická, idempotentní a čistá vzhledem ke vstupnímu stavu.

[Odkaz na 2.5. Co dělat, když to nejde napsat.]

INV-03 Každá data-touching migrace ověřuje vlastní úplnost.

[Post-condition tvaru „zbývá nula“, nikoli „update proběhl“.]

INV-04 Baseline a plný řetěz migrací vedou k identickému schématu.

[Odkaz na 2.6. Ověřuje se schema diffem v CI.]

INV-05 Poslední patch v řadě je jediným podporovaným vstupem do dalšího majoru.

[Zúžení vstupní brány. Vynucuje instalátor, nikoli dokumentace.]

INV-06 Destruktivní změna se nikdy neprovádí v jednom vydání.

[Expand/contract napříč verzemi. Bod bez návratu neexistuje.]

4 Mechanismy

[Stub. Každý mechanismus explicitně uvádí, který invariant vynucuje a jakou divergenci zkracuje. Kandidáti:]

- *Rozdělení migrace na preflight a mutate.* Read-only ověření předpokladů spustitelné kdykoli, i měsíce před vydáním.
- *Expand/contract napříč vydáními.*
- *Opravy jako idempotentní repeatable skripty* namísto položek v lineárním řetězu.

- *Konvergenční uzávěrka* na konci patch řady.
- *Schema diff v CI* přes všechny reálné cesty.
- *Profil instance jako telemetry* — tvary dat, nikoli data.
- *Kohorty (rings)* pro postupné uvolňování.

5 Rozhodnutí, která zůstávají úsudkem

[Stub. Nikoli pravidla, ale vyjmenované veličiny, které se váží. Cílem kapitoly je odlišit úsudek nad známými veličinami od intuice. Kandidáti: hotfix versus odklad do nejbližšího vydání; prerekvizitní patch versus řešení uvnitř nového majoru; doba běhu migrace proti odstávkovému oknu u největší instance.]

6 Antipatterny

[Stub. Squash bez zachování řetězu pro upgrade cestu. Ruční zásah u zákazníka. On-prem s jedinou odchozí závislostí. Nulový počet hotfixů jako cíl namísto výsledku.]

Zdroje

[Reference se zapojují přímo do textu u příslušných invariantů, nikoli do samostatného appendixu. Nashromážděné:]

- GitLab, charts/gitlab issue #4522 — vynucení poslední minor verze předchozí série jako vstupu do nového majoru. Nezávislý doklad k INV-06 (viz sekce 3).
- GitLab, container-registry issue #1516 — rozbitý migrační stav při přeskočení post-deployment migrací.
- Langfuse, self-hosting docs — schéma a migrace verzované jako jeden celek; úprava schématu přesouvá odpovědnost na zákazníka.
- Ardent Performance, *Database Schema Migrations in 2026* (survey) — srovnání přístupů u Sentry, Zulip a dalších.
- Tenable, *Software Release Lifecycle Policy* — kadence a závazek k dostupnosti upgrade cest.
- Microsoft, *Finance + Operations (on-premises) version update policy* — rozlišení kritických a nekritických aktualizací.
- R. Mironov, *Product Sprawl* — popis symptomu z produktové perspektivy.